

CoSc 3311

Fundamentals of Software Engineering

Dagmawi Lemma (PhD)

Assistant Professor, AAU

courses.dag@gmail.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

1

5. Implementation and Testing

CoSc3311 – Fundamentals of Software Engineering

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

239

Coding Standard

- Once the design of the system is completed and after each subsystem is well specified the next step is to convert each subsystem (module) specification into runnable code
- As we are working as a team, we need to be able to manage the various programming culture/style in accordance the selected programming language, hence we need to have **coding standard**

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

240

Coding Standard

- **Coding Standard** is a set of guidelines often derived from what is found to be best practice.
 - It is mainly about programming styles and conventions that developers adhere to when writing source code .
- Often includes
 - Naming standard
 - Whitespace
 - Indentation
 - Comments (internal documentation)
 - Use of parentheses
 - etc.
- Therefore, before implementation (writing the codes) the programming team shall agree on a certain coding standard.
 - That can be developed in-house, or adapted from existing standards (e.g., pears for php)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

241

Software Development Framework

- During implementation, most part of the codes in various module could be similar except slightly different due to the functional requirements of the particular module.

<pre><?php \$ machine = "LibraryServer"; \$ user = "LibAdmin "; \$ password = "xyz"; \$ dbname = "NibabDB"; // Create connection \$conn = new mysqli(\$ machine, \$ user, \$ password, \$ dbname); // Check connection if (\$conn->connect_error) { die("Connection failed: " . \$conn->connect_error); } \$sql = " INSERT INTO Member (firstname, lastname, email) VALUES ('Abebe', 'Alemu', 'abebe@example.com)"; if (\$conn->query(\$sql) === TRUE) { echo "New record created successfully"; } else { echo "Error: " . \$sql . "
". \$conn->error; } \$conn->close(); ?></pre> 	<pre><?php \$ machine = "LibraryServer"; \$ user = "LibAdmin "; \$ password = "xyz"; \$ dbname = "NibabDB"; // Create connection \$conn = new mysqli(\$ machine, \$ user, \$ password, \$ dbname); // Check connection if (\$conn->connect_error) { die("Connection failed: " . \$conn->connect_error); } \$sql = " INSERT INTO Book (isbn, title, author, publisher, nocopies) VALUES ('9783639210057', ' How to Build Malicious Process Conscious Operating System ', ' Dagmawi Lemma; Mulugeta Lebssie ', ' VDM', 5)"; if (\$conn->query(\$sql) === TRUE) { echo "New record created successfully"; } else { echo "Error: " . \$sql . "
". \$conn->error; } \$conn->close(); ?></pre> 
--	---

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

242

Software Development Framework

- Also, it is common to reuse previous codes/components.
 - Especially general purpose codes (e.g., creating database connection, file operation, authentication module, etc.) are often reused (with minimal modification) from previous projects.

<pre><?php \$ machine = "LibraryServer"; \$ user = "LibAdmin "; \$ password = "xyz"; \$ dbname = "NibabDB"; // Create connection \$conn = new mysqli(\$ machine, \$ user, \$ password, \$ dbname); // Check connection if (\$conn->connect_error) { die("Connection failed: " . \$conn->connect_error); } \$sql = " INSERT INTO Member (firstname, lastname, email) VALUES ('Abebe', 'Alemu', 'abebe@example.com)"; if (\$conn->query(\$sql) === TRUE) { echo "New record created successfully"; } else { echo "Error: " . \$sql . "
". \$conn->error; } \$conn->close(); ?></pre> 	<pre><?php \$ machine = "192.168.5.14"; \$ user = "Daniel"; \$ password = "123 "; \$ dbname = " payroll "; // Create connection \$conn = new mysqli(\$ machine, \$ user, \$ password, \$ dbname); // Check connection if (\$conn->connect_error) { die("Connection failed: " . \$conn->connect_error); } \$sql = " INSERT INTO Employee (firstname, lastname, salary) VALUES ('Aster', 'Yohanes', 40000)"; if (\$conn->query(\$sql) === TRUE) { echo "New record created successfully"; } else { echo "Error: " . \$sql . "
". \$conn->error; } \$conn->close(); ?></pre> 
--	---

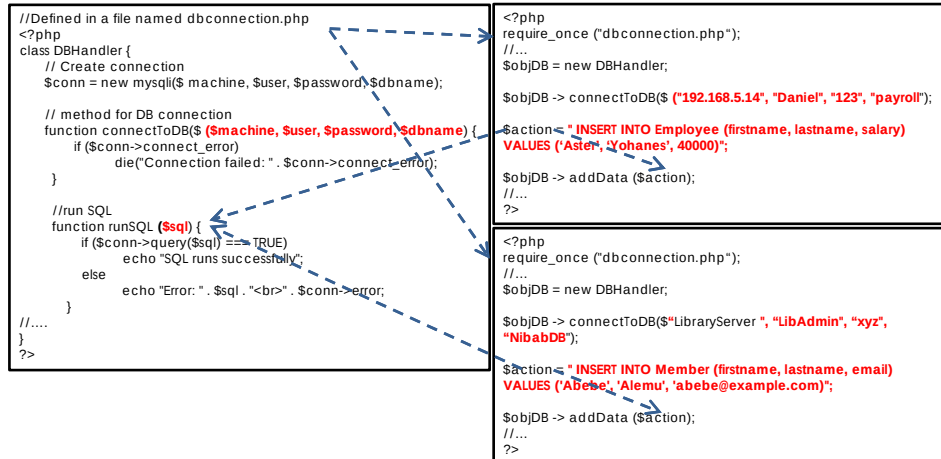
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

243

Software Development Framework

- If we have common code definition as component, then we can add flexibility and efficiency in the implementation



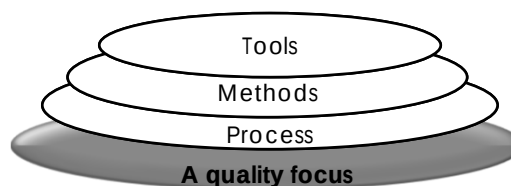
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

244

Concept of Software Framework

- Pressman indicates that "The bedrock that supports software engineering is a quality focus" (Pressman, 2001).



- The quality of a product is not only equated to the quality of the process but also to the tools and the methods we apply.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

245

Concept of Software Framework

- One method we can apply to improve the quality of the software is using a framework
 - i.e. mainly from the development perspective
- Framework is meant to improve efficiency of creating new software,
 - Reliability and robustness of new software can be also improved
 - Developers can devote their time to meet the requirements rather than dealing with the common and standard details, hence developers productivity is improved
 - how data is stored, protected, retrieved, uploaded, etc. are often standard low level functionalities

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

246

Framework Vs Libraries

- Reusability! Yes
- A Software Framework
 - is generic (but can be specific to some development platform or domain)
 - is reusable software platform
 - Includes code libraries, APIs, compilers, support programs and tools

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

247

Framework Vs Libraries

- But it is different from libraries (e.g. OO library classes) in
 - inversion control
 - if a library is used, the objects and methods implemented by the library are instantiated and invoked by the custom application being developed
 - Knowledge about the objects to be instantiated and the methods to be called is required to achieve the goal
 - If a framework is used, the objects and methods custom to the application is implemented and these objects and methods are invoked by the framework, so the flow of control is not controlled by the caller but by the framework

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

248

Framework Vs Libraries

- But it is different from libraries (e.g. OO library classes) in (... *continued*)
 - Extensibility (hot spot)
 - overriding is a common way to customize framework behavior. (i.e. framework classes can be overridden in user-defined code)
 - programmers can add their code to add functionality
 - default behavior
 - can be useful
 - non-modifiable (frozen spot)
 - The framework code is not allowed to be modified but can be extended

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

249

Challenges

- Code bloat
 - increased program size
 - extensibility Vs maintainability

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

250

The *Myth*

- Less development time
 - only true if the developers have the experience with **framework x** and if using **framework x**
 - The learning curve
- useful to attain reliability
 - often true, but can we trust the source? do we really know the framework in its entirety?
 - there could be security concern

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

251

The *Myth*

- generalization
 - usually effective frameworks are those evolve from the common code of applications in a certain domain
 - difficult to have “one-size-fits-all” type

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

252

The Framework Zoo

- Top ten Java frameworks observed in customer applications
 - Spring MVC
 - Struts 1.x
 - Apache Axis
 - Apache Xerces
 - Hibernate
 - JDOM
 - Java Applet
 - Apache Velocity
 - Apache ORO
 - JAX-W5

www.veracode.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

253

The Framework Zoo

PHP Framework	PHP4	PHP5	MVC	Multiple DB's	DB Objects	Templates	Caching	Validation	Ajax
Akeelos 🇳🇮 🇵🇪	✓	✓	✓	✓	✓	✓	✓	✓	✓
ash.MVC 🇳🇮	-	✓	✓	-	✓	✓	-	✓	-
CakePHP 🇳🇮 🇵🇪	✓	✓	✓	✓	✓	-	✓	✓	✓
CodeIgniter 🇳🇮 🇵🇪	✓	✓	✓	✓	✓	✓	✓	✓	-
DIY 🇳🇮	-	✓	✓	-	✓	✓	✓	-	✓
eZ Components 🇳🇮	-	✓	-	✓	✓	✓	✓	✓	-
Fusebox 🇳🇮	✓	✓	✓	✓	-	-	✓	-	✓
PHP on TRAX 🇳🇮	-	✓	✓	✓	✓	-	-	✓	✓
PHPDevShell 🇳🇮	-	✓	✓	-	✓	✓	✓	✓	✓
PhpOpenbiz 🇳🇮	-	✓	✓	✓	✓	✓	-	✓	✓
Prado 🇳🇮 🇵🇪	-	✓	✓	✓	✓	✓	✓	✓	✓
QPHP 🇳🇮	✓	✓	✓	✓	✓	✓	-	✓	✓
Seagull 🇳🇮	✓	✓	✓	✓	✓	✓	✓	✓	✓
Symfony 🇳🇮 🇵🇪	-	✓	✓	✓	✓	-	✓	✓	✓
WACT 🇳🇮 🇵🇪	✓	✓	✓	✓	✓	✓	-	✓	-
WASP 🇳🇮	-	✓	✓	-	✓	✓	-	✓	✓
Yii 🇳🇮 🇵🇪	-	✓	✓	✓	✓	✓	✓	✓	✓
Zend 🇳🇮 🇵🇪	-	✓	✓	✓	✓	✓	✓	✓	✓
ZooP 🇳🇮	✓	✓	✓	✓	✓	✓	✓	✓	✓

www.phpframeworks.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

254

Testing

- How can we assure the coding standard has been followed?
 - Note that compilers may not help you
- How can we assure that the specification we provide in the design and analysis are correctly implemented?
- How can we assure even the analysis and the design are coherently defined?

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

255

What is Testing?

- Testing is the process of analyzing a system or system component to detect the differences between specified (required) and observed (existing) behavior.
 - Design Specification Vs Implementation
 - User Requirement Vs existing service provide by the system
 - Required Vs exhibited behavior of the system
 - Source code style Vs Selected coding standard
 - etc.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

256

Goal of Testing

- Since testing is a systematic attempt to find **errors** in *a planned way in the implemented software*, the explicit goal is to demonstrate the presence of faults
- Error € manifests fault
 - Suppose there is a programming defect (fault), then the program will have an effect (output) different from what has been specified in the design; and the difference between the actual output and the desired output is the error. Thus:

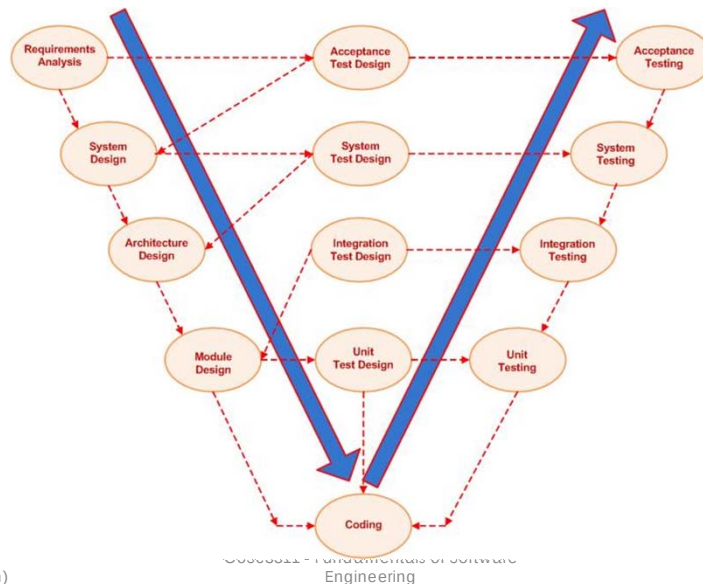
$$e = |x_{\text{designed}} - x_{\text{measured}}|$$

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

257

The V-model



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software Engineering

258

Testing Concepts

- A **fault**, also called bug or defect, is a design or coding mistake that may cause abnormal component behavior.
- An **error** is a manifestation of a fault during the execution of the system.
- A **failure** is a deviation between the specification of a component and its behavior. A failure is triggered by one or more errors.
- A **correction** is a change to a component. The purpose of a correction is to repair a fault. Note that a correction can introduce new faults

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software Engineering

259

Testing Concepts

- A **component** is a part of the system that can be isolated for testing. A component can be an object, a group of objects, or one or more subsystems.
- A **test stub** is a partial implementation of components on which the tested component depends
- A **test driver** is a partial implementation of a component that depends on the tested component. Test stubs and drivers enable components to be isolated from the rest of the system for testing

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

260

Testing Concepts

Very simple example

Problem:

- finding volume of cylinder

Solution:

- Product of the height and area of circle (the base)

$$Volume(h, r) = h \times Area(r)$$

$$Area(r) = \pi \times Square(r)$$

$$Square(r) = r \times r$$

If A(r) is completed and selected for testing, then

- Either we have to wait until both V(h,r) and S(r) are completed since A(r) has no use by itself, or
- V(h,r) and S(r) shall be defined as *drive* and *stub* respectively

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

261

Testing Concepts

Very simple example (continued)

```
double volume(double h, double r)
{
    return h x area(r);
}
```

```
double square(double r)
{
    return r x r;
}
```

```
double area(double r)
{
    //this function returns area of a circle with radius r
    const double PI = 3.14; //this constant can be global as well
    double result = 0; //variable for storing the result
    double sq= 0; //variable for storing square of a number
    sq = square (r); //calculating square of a number
    result = PI x sq; // calculating area
    return result;
}
```

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

262

Testing Concepts

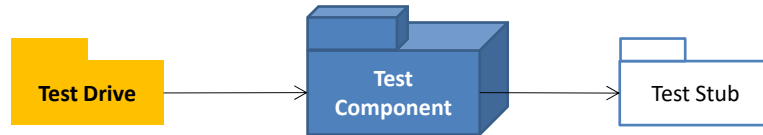
- At the analysis stage the focus is on the problem domain while at the design stage the focus is on the solution domain
 - To provide the solution the system is decomposed into subsystems and
 - each subsystem is defined by the service it provides – hence cohesively defined;
 - the subsystems are coupled;
 - Yet from testing perspective,
 - Either we have to wait for testing until all the subsystems are completely implemented – which is not desired
 - Or we have to isolate the subsystem, which is completely implemented, for testing
 - However, since the one being isolated is coupled with other subsystems, then those shall be defined as stab and drive

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

263

Testing Concepts



```

import Shops.PizzaHouse;
class Order{
  /* this class is one of the classes in the Shops package and it may not be
  complete but can be defined partially to invoke a component under test*/
  PizzaHouse myFavorite = new PizzaHouse;
  myFvorite.showshippingprice(myAddress);
  //...
}
  
```

Drive

Depends on

```

class PizzaHouse{
  //...
  Public static void showshippingprice(Location address){
    /*suppose this is the method to be tested, then it
    must be completely implemented*/
  }
  //...
}
  
```

Test
Comp

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

264

Testing Concepts



```

import ServiceProviders.Courier
class PizzaHouse{
  //...
  Public static void showshippingprice(Location address){
    /*while completely implemented, still may depend on others*/
    Courier dispatchArranged = new Courier;
  }
  //...priceList=dispatchArranged.getPrice(address);
  //...
}
  
```

Test
Comp

Depends on

```

class Courier{
  /* this class is one of the classes in the ServiceProvider package and it may not be complete but can be
  defined partially to provide price List*/
  priceList getPrice(Location address){
    //...;
  }
  //...
}
  
```

Stub

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

265

Testing Concepts

- Test Case
 - It is a set of **inputs** and **expected results** that exercises a component with the **purpose of causing failures** and **detecting faults**
 - Selection of test case is dictated by the objective of the test plan
 - For example
 - Are we having structural or functional testing
 - If we have structural testing, then which coverage metrics is our focus?

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

266

Testing Concepts

- Test Case (Continued)
 - Scenarios (evaluation scenarios) are primary sources for test cases
 - Also each test case may appear with various scenario
 - When a component is tested using test drive, the test case is used at the test drive
 - That is the input is feed through the drive
 - The actual result is received at the drive
 - And the one who is comparing the actual (measured) result with the expected result is known as oracle

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

267